NAME : AKBARI SMIT C.

ROLL NO. : 2

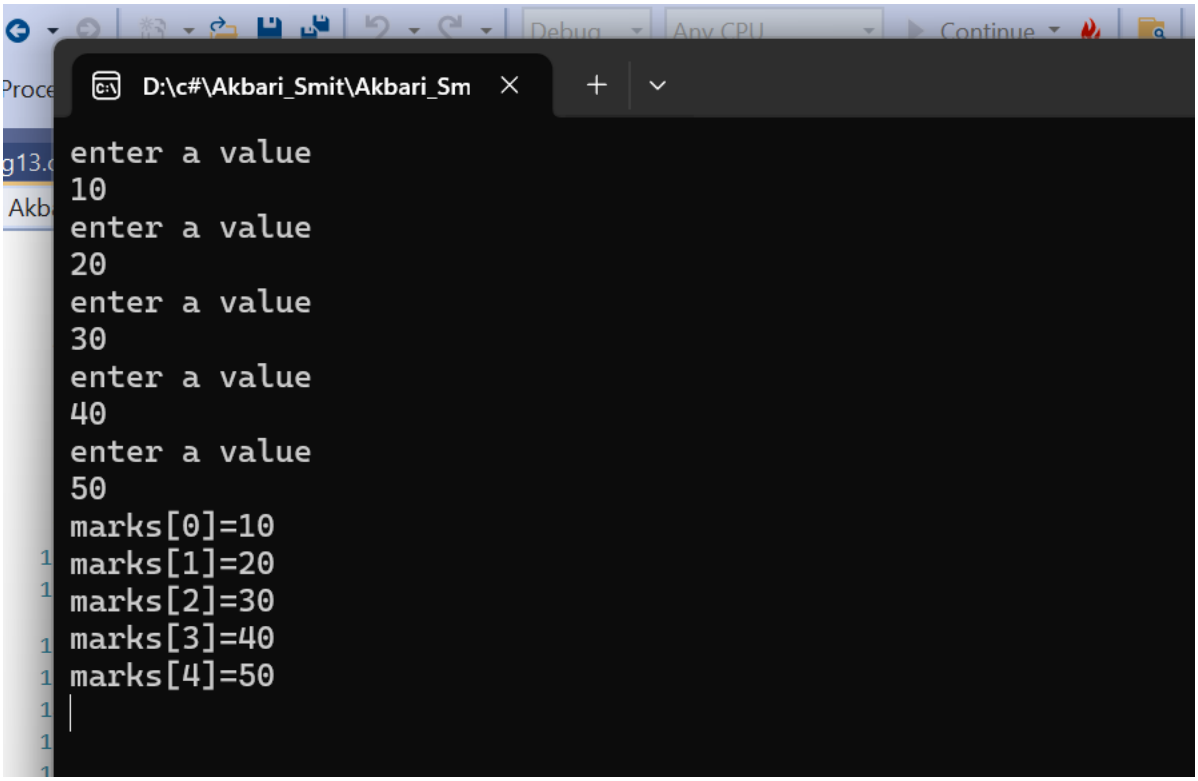
DIVISION : C5

ENROLLMRNT NO. : 240801016

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Akbari_Smit
{
    2 references
    class prg1
    {
        // accept five numbers in an array and dispaly it.
        1 reference
        public void prg()
        {
            int[] marks = new int[5];

            for (int i = 0; i < 5; i++)
            {
                Console.WriteLine("enter a value");
                marks[i] = Convert.ToInt32(Console.ReadLine());
            }
            for (int i = 0; i < 5; i++)
            {
                Console.WriteLine("marks[{0}]={1}", i, marks[i]);
            }
        }
    }
}
```



```
using System.Text;
using System.Threading.Tasks;

namespace Akbari_Smit
{
    2 references
    class prg2
    {
        //accept 10 integer and find sum and average.
        1 reference
        public void prg()
        {
            int[] num = new int[10];
            int sum = 0;


            for (int i = 0; i < num.Length; i++)
            {
                Console.WriteLine("enter a value");
                num[i] = Convert.ToInt32(Console.ReadLine());
            }

            for (int i = 0; i < num.Length; i++)
            {

                sum = sum + num[i];
            }

            Console.WriteLine("Sum of the numbers: " + sum);
            double avg = (double)sum / num.Length;
            Console.WriteLine("Average of the numbers: " + avg);
        }
    }
}
```

```
D:\c#\Akbari_Smit\Akbari_Sm
enter a value
1
enter a value
2
enter a value
3
enter a value
4
enter a value
5
enter a value
6
enter a value
7
enter a value
8
enter a value
9
enter a value
1
Sum of the numbers: 46
Average of the numbers: 4.6
```

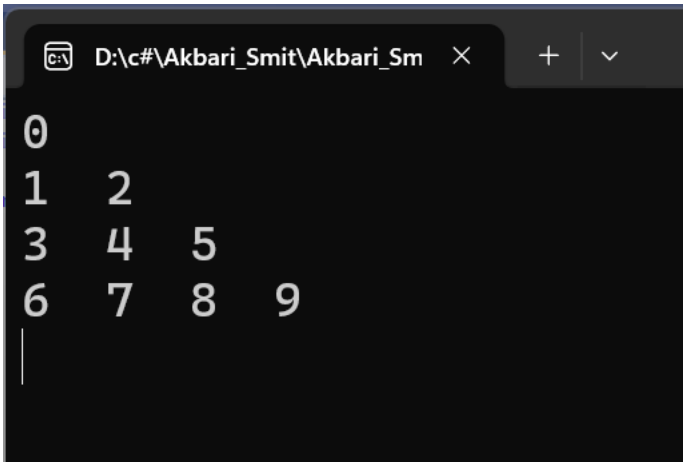
```
namespace Akbari_Smit
{
    2 references
    class prg3
    {
        //display the following (use jagged array)
        1 reference
        public void prg()
        {
            int rows = 4;
            int[][] pattern = new int[rows][];

            int num = 0;

            for (int i = 0; i < rows; i++)
            {
                pattern[i] = new int[i + 1];

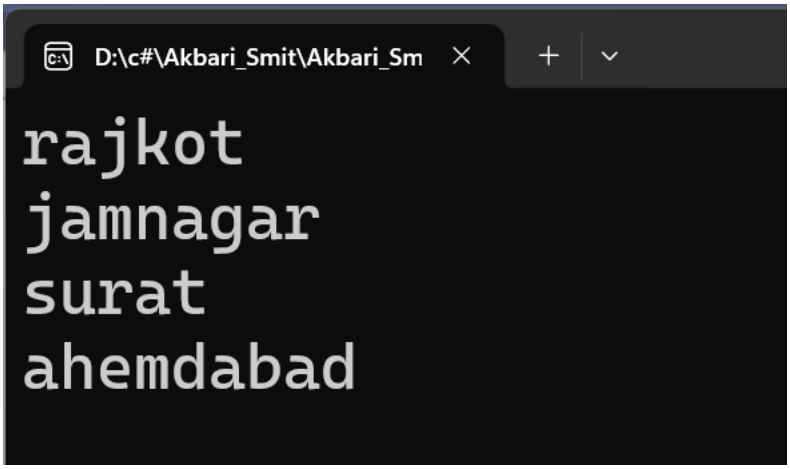
                for (int j = 0; j < pattern[i].Length; j++)
                {
                    pattern[i][j] = num++;
                    Console.Write(pattern[i][j] + "  ");
                }

                Console.WriteLine();
            }
        }
    }
}
```



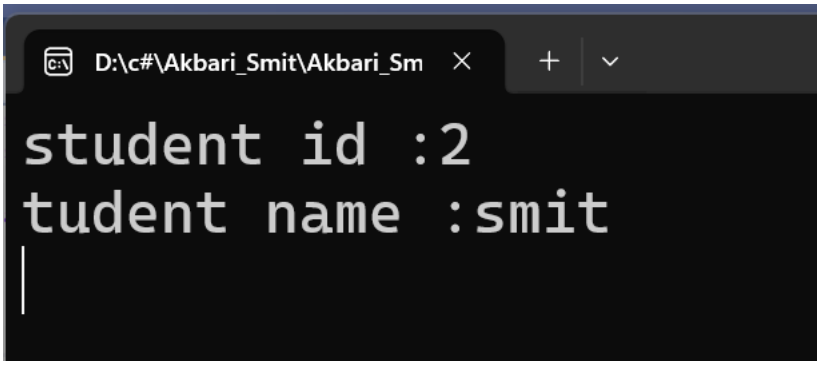
```
namespace Akbari_Smit
{
    2 references
    class prg4
    {
        //write a program to display array elements using foreach loop
        1 reference
        public void prg()
        {
            string[] city = { "rajkot", "jamnagar", "surat", "ahemdabad" };

            foreach (string cities in city)
            {
                Console.WriteLine(cities);
            }
        }
    }
}
```



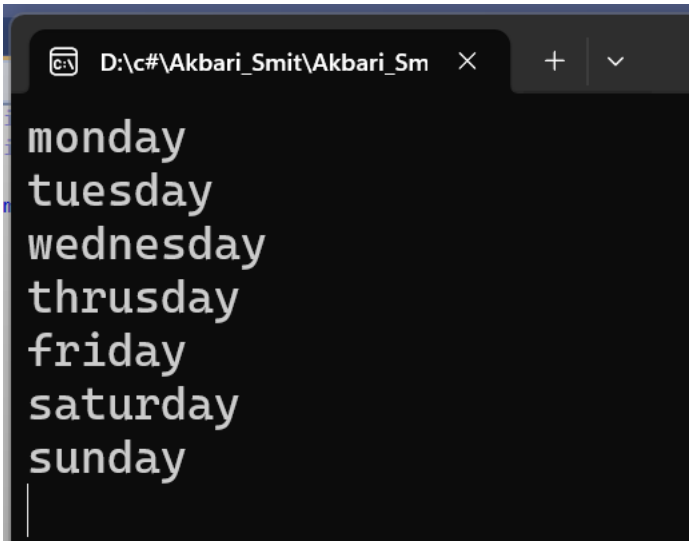
```
namespace Akbari_Smit
{
    2 references
    class prg5
    {
        //write a program using structure to store and display student details
        1 reference
        struct stud
        {
            public int id;
            public string nm;
        }
        1 reference
        public void prg()
        {
            stud s;
            s.id = 2;
            s.nm = "smit";

            Console.WriteLine("student id :" + s.id);
            Console.WriteLine("tudent name :" + s.nm);
        }
    }
}
```

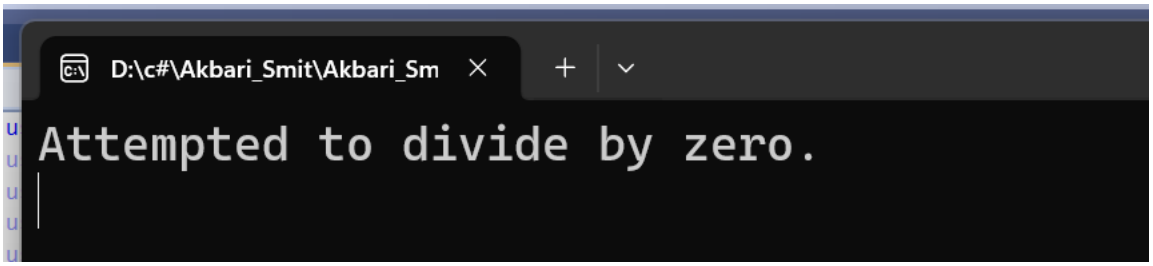


```
namespace Akbari_Smit
{
    //write a program using enumeration to display days of week

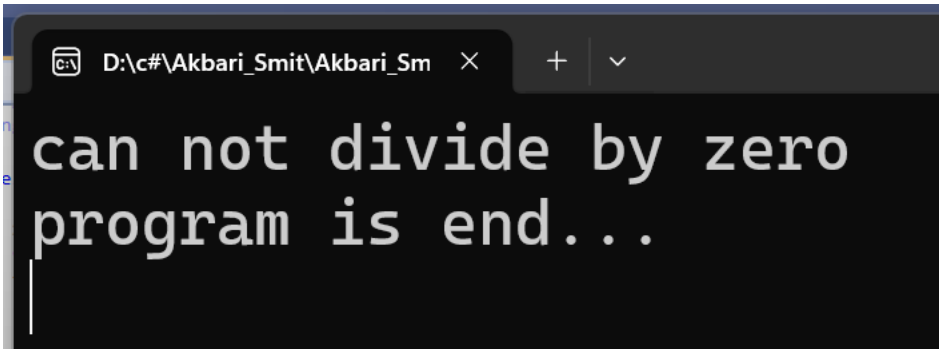
    1 reference
    enum days
    {
        monday,
        tuesday,
        wednesday,
        thrusday,
        friday,
        saturday,
        sunday
    }
    2 references
    class prg6
    {
        1 reference
        public void prg()
        {
            for (int i = 0; i <= 6; i++)
            {
                Console.WriteLine((days)i);
            }
        }
    }
}
```



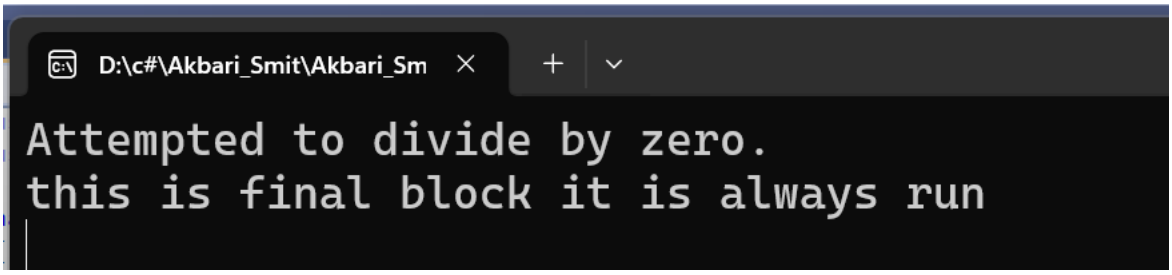
```
namespace Akbari_Smit
{
    2 references
    class prg7
    {
        //write program to handle divide-by-zero exception using try catch
        1 reference
        public void prg()
        {
            try
            {
                int a = 10;
                int b = 0;
                int c = a / b;
            }
            catch (DivideByZeroException e)
            {
                Console.WriteLine(e.Message);
            }
        }
    }
}
```



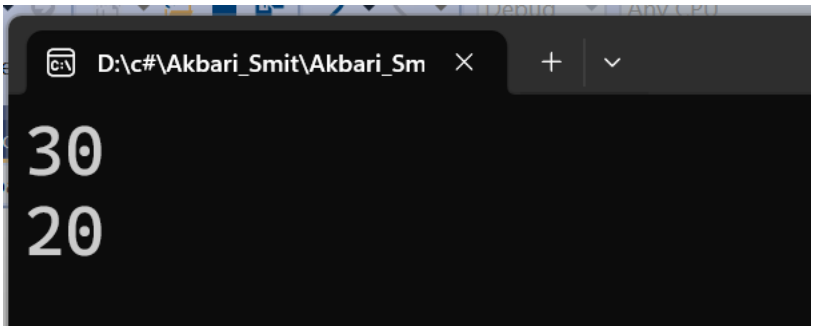
```
namespace Akbari_Smit
{
    2 references
    class prg8
    {
        //write a programe to handle multiple exception in c#
        1 reference
        public void prg()
        {
            try
            {
                int a = 30;
                int b = 0;
                Console.WriteLine("division is"+a/b);
                int[] arr = { 11, 22, 33 };
                Console.WriteLine(arr[2]);
            }
            catch(DivideByZeroException)
            {
                Console.WriteLine("can not divide by zero");
            }
            catch(IndexOutOfRangeException)
            {
                Console.WriteLine("out of index...please enter eligible index");
            }
            finally
            {
                Console.WriteLine("program is end...");
            }
        }
    }
}
```



```
namespace Akbari_Smit
{
    2 references
    class prg9
    {
        //write a programe to use try-catch-finally block.
        1 reference
        public void prg()
        {
            try
            {
                int a = 10;
                int b = 0;
                int c = a / b;
            }
            catch (DivideByZeroException e)
            {
                Console.WriteLine(e.Message);
            }
            finally
            {
                Console.WriteLine("this is final block it is always run");
            }
        }
    }
}
```



```
namespace Akbari_Smit
{
    2 references
    class prg10
    {
        //create method that will interchange the value of its parameters using by reference
        0 references
        public void prg(ref int a, ref int b)
        {
            int c;
            c = a;
            a = b;
            b = c;
            Console.WriteLine(a);
            Console.WriteLine(b);
        }
    }
}
```

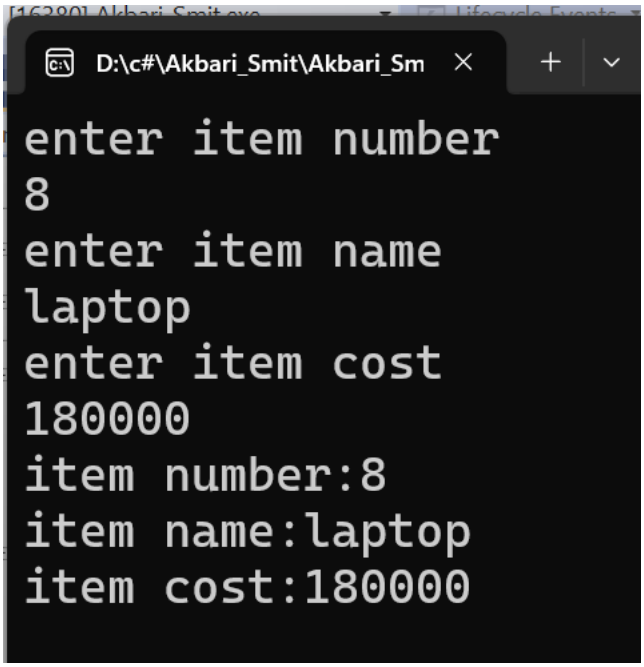


```
namespace Akbari_Smit
{
    /*Create a class named Item that has itemno, itemname, and itemcost as data members.
      Add GetItem() and PutItem() as member functons.Write an appropriate Main()method.*/
    2 references
    class item
    {
        int itemno;
        string itemname;
        int itemcost;

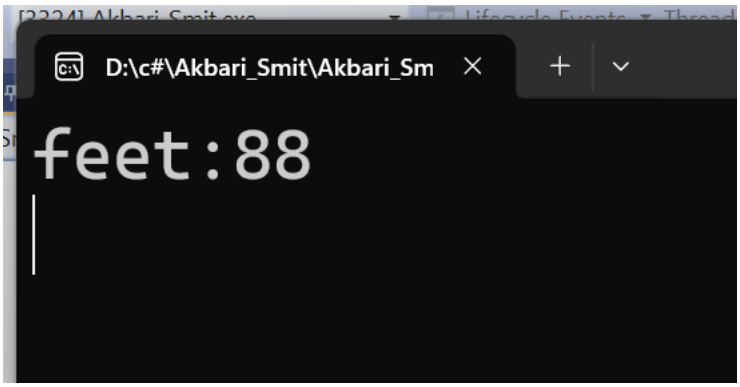
        1 reference
        public void getitem()
        {
            Console.WriteLine("enter item number");
            itemno = Convert.ToInt32(Console.ReadLine());

            Console.WriteLine("enter item name");
            itemname = Console.ReadLine();

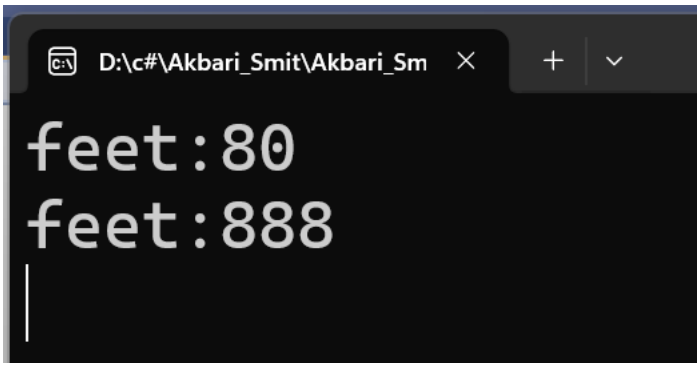
            Console.WriteLine("enter item cost");
            itemcost = Convert.ToInt32(Console.ReadLine());
        }
        1 reference
        public void putitem()
        {
            Console.WriteLine("item number:" + itemno);
            Console.WriteLine("item name:" + itemname);
            Console.WriteLine("item cost:" + itemcost);
        }
    }
}
```



```
namespace Akbari_Smit
{
    //Create a class named Distance that has feet as a data member. Create a parameterless constructor to initialize feet to zero.
    3 references
    class distance
    {
        int feet;
        1 reference
        public distance()
        {
            feet = 88;
        }
        1 reference
        public void display()
        {
            Console.WriteLine("feet:" + feet);
        }
    }
}
```

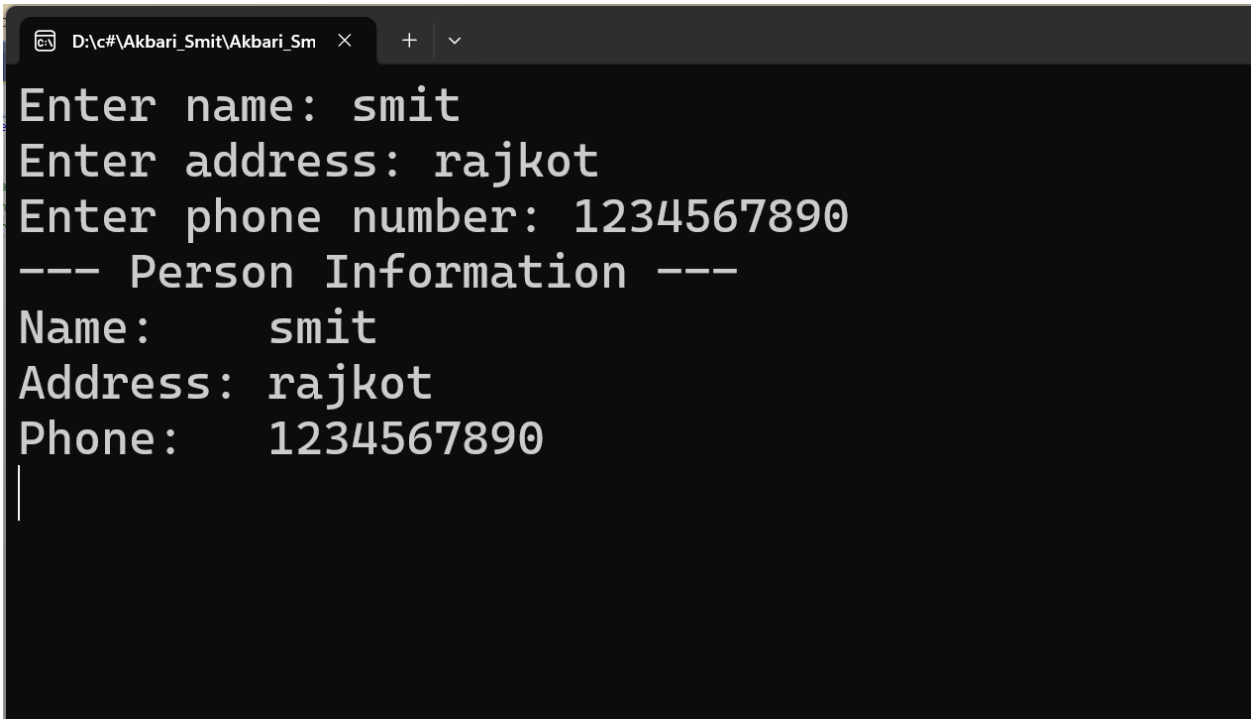


```
namespace Akbari_Smit
{
    //Modify the above program by adding another constructor that accepts an integer argument to initialize feet.
    2 references
    class distances
    {
        int feet;
        0 references
        public distances()
        {
            feet = 80;
        }
        0 references
        public distances(int intilial)
        {
            feet = intilial;
        }
        0 references
        public void display()
        {
            Console.WriteLine("feet:" + feet);
        }
    }
}
```



```
namespace Akbari_Smit
{
    /*Write a class named Person with private data members name, address, and phone. Add
a parameterized constructor Person(n, a, p) to iniθalize these data members. Add a
method named Display() to display the person informaθon. Accept the values from the
user and pass them to the constructor while creating the objects.*/
    3 references
    class person
    {
        private string name;
        private string address;
        private string phone;

        1 reference
        public person(string n, string a, string p)
        {
            name = n;
            address = a;
            phone = p;
        }
        1 reference
        public void Display()
        {
            Console.WriteLine("--- Person Information ---");
            Console.WriteLine($"Name:    {name}");
            Console.WriteLine($"Address: {address}");
            Console.WriteLine($"Phone:   {phone}");
        }
    }
}
```



```
namespace Akbari_Smit
{
    //Add the destructor in the above program with the name ~Person()
    4 references
    class persons
    {
        private string names;
        private string addresss;
        private string phones;

        1 reference
        public persons(string n, string a, string p)
        {
            names = n;
            addresss = a;
            phones = p;
        }
        1 reference
        public void Display()
        {
            Console.WriteLine("--- Person Information ---");
            Console.WriteLine($"Name:    {names}");
            Console.WriteLine($"Address: {addresss}");
            Console.WriteLine($"Phone:   {phones}");
        }
        0 references
        ~persons()
        {
            Console.WriteLine($"destructor calles for :{names}");
        }
    }
}
```

```
D:\c#\Akbari_Smit\Akbari_Sm
Enter name: smit
Enter address: rajkot
Enter phone number: 123243123
--- Person Information ---
Name:    smit
Address: rajkot
Phone:   123243123
```

```
namespace Akbari_Smit
{
    /*Define a class named student having data members name, age, gender (m/f) and
various overloaded constructors for accep0ng the details of student. Add display
method to display informa0on of student.*/
    9 references
    class Student
    {
        private string name;
        private int age;
        private char gender;

        1 reference
        public Student()
        {
            name = "Unknown";
            age = 0;
            gender = 'U';
        }

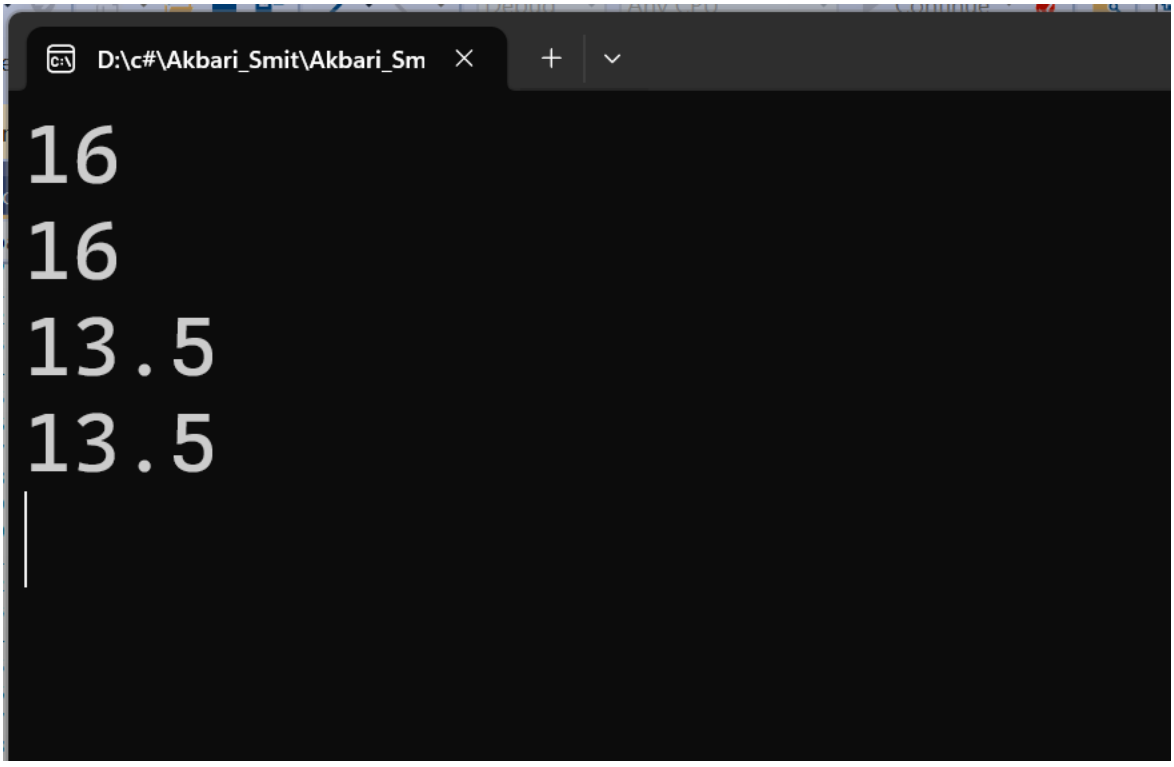
        1 reference
        public Student(string studentName, int studentAge)
        {
            name = studentName;
            age = studentAge;
            gender = 'U'; // Unspecified
        }

        1 reference
        public Student(string studentName, int studentAge, char studentGender)
        {
            name = studentName;
            age = studentAge;
            gender = studentGender;
        }

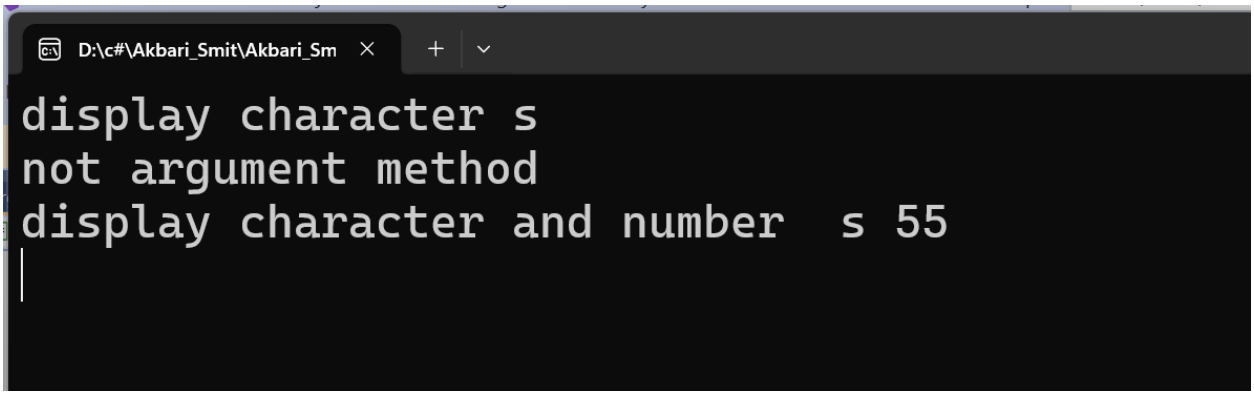
        3 references
        public void Display()
        {
            Console.WriteLine("--- Student Information ---");
            Console.WriteLine($"Name   : {name}");
            Console.WriteLine($"Age    : {age}");
            Console.WriteLine($"Gender : {gender}");
        }
    }
```

```
D:\c#\Akbari_Smit\Akbari_Sm
--- Student Information ---
Name   : Unknown
Age    : 0
Gender : U
--- Student Information ---
Name   : Alice
Age    : 20
Gender : U
--- Student Information ---
Name   : Bob
Age    : 22
Gender : M
```

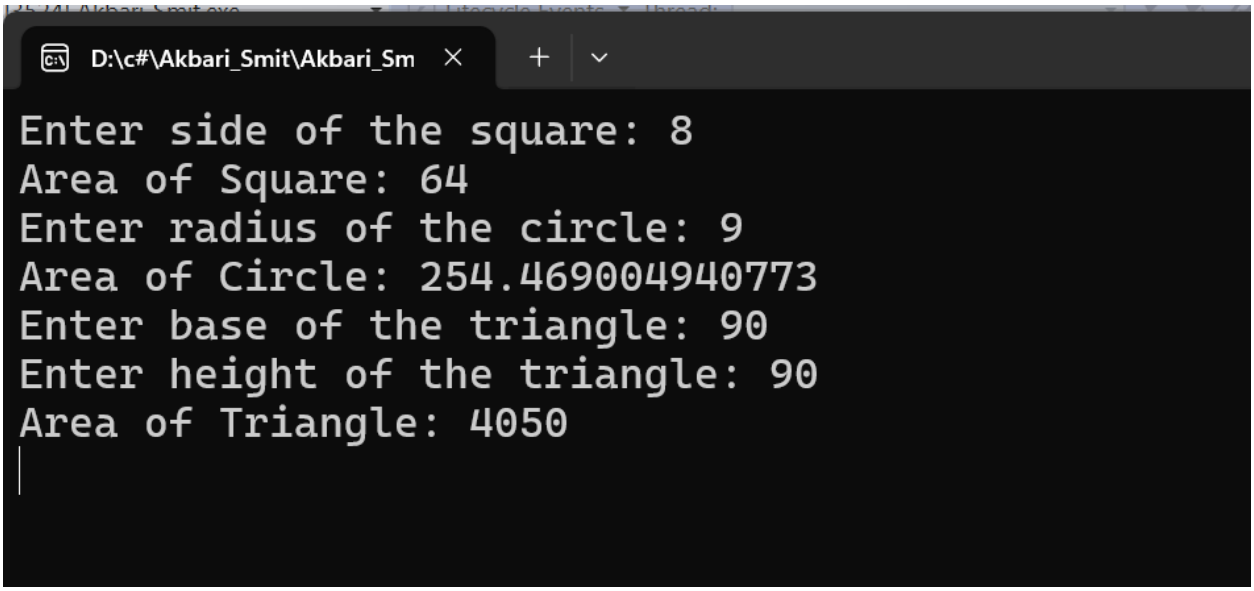
```
namespace Akbari_Smit
{
    /*Create four methods having the same name Sum.
a. First method will accept two int arguments and return int value.
b. Second will accept two float arguments and return float value.
c. Third will accept one int and one float argument and return float value.
d. Fourth will accept one float and one int argument and return float value.*/
    2 references
    class calculator
    {
        1 reference
        public int sum(int a, int b)
        {
            return a + b;
        }
        1 reference
        public float sum(float a, float b)
        {
            return a + b;
        }
        1 reference
        public float sum(int a, float b)
        {
            return a + b;
        }
        1 reference
        public float sum(float a, int b)
        {
            return a + b;
        }
    }
}
```



```
namespace Akbari_Smit
{
    /*Create three methods having same name Display
a. First will accept one char and one int value and display the two.
b. Second will accept one char as argument and display it.
c. Third will not accept any argument and display no argument.*/
    2 references
    class printer
    {
        1 reference
        public void display(char c, int n)
        {
            Console.WriteLine("display character and number  "+ c+ " " + n);
        }
        1 reference
        public void display(char c)
        {
            Console.WriteLine("display character "+c);
        }
        1 reference
        public void display()
        {
            Console.WriteLine("not argument method");
        }
    }
}
```

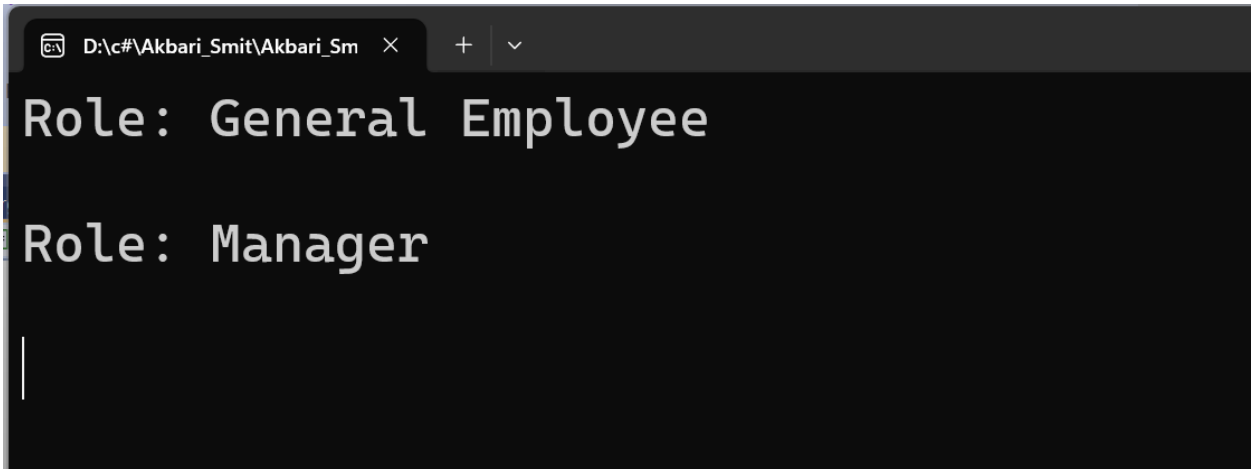


```
namespace Akbari_Smit
{
    /*Create a method called Area which is used to calculate area of triangle, square, and
circle. Write a main method that accepts value from user for calculaθng area of all
the shapes.*/
    0 references
    public class ShapeAreaCalculator
    {
        0 references
        public double Area(double side)
        {
            return side * side;
        }
        0 references
        public double Area(float radius)
        {
            return Math.PI * radius * radius;
        }
        0 references
        public double Area(double b, double h)
        {
            return 0.5 * b * h;
        }
    }
}
```



```csharp
namespace Akbari_Smit
{

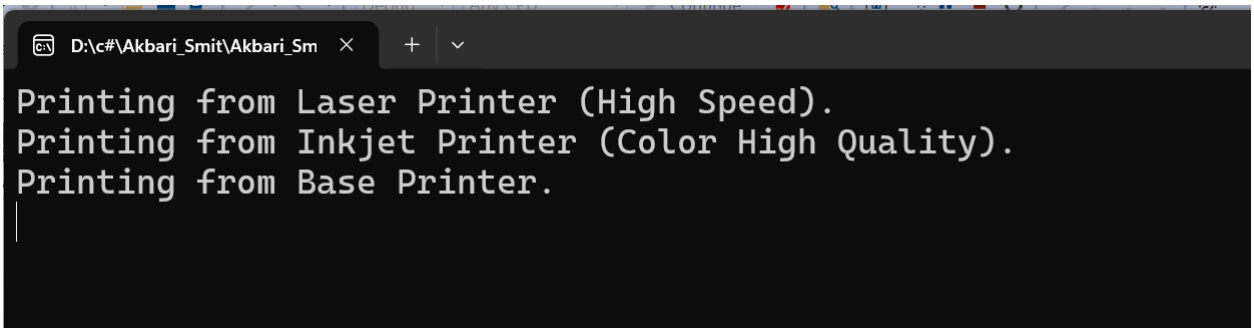
    //Create a base class Employee and derived class Manager which overrides a DisplayInfo() method to show method overriding.
    1 reference
    class Employee
    {
        1 reference
        public virtual void DisplayInfo()
        {
            Console.WriteLine("Role: General Employee");
        }
    }
    0 references
    class Manager : Employee
    {
        1 reference
        public override void DisplayInfo()
        {
            Console.WriteLine("Role: Manager (Derived Class)");
        }
    }
}
```



```
namespace Akbari_Smit
{
    /*Demonstrate run0me polymorphism using a virtual method Print() in the base
class and override it in two derived classes.*/
    6 references
    class BasePrinter
    {
        5 references
        public virtual void Print()
        {
            Console.WriteLine("Printing from Base Printer.");
        }
    }

    1 reference
    class LaserPrinter : BasePrinter
    {
        4 references
        public override void Print()
        {
            Console.WriteLine("Printing from Laser Printer (High Speed).");
        }
    }

    1 reference
    class InkjetPrinter : BasePrinter
    {
        4 references
        public override void Print()
        {
            Console.WriteLine("Printing from Inkjet Printer (Color High Quality).");
        }
    }
}
```

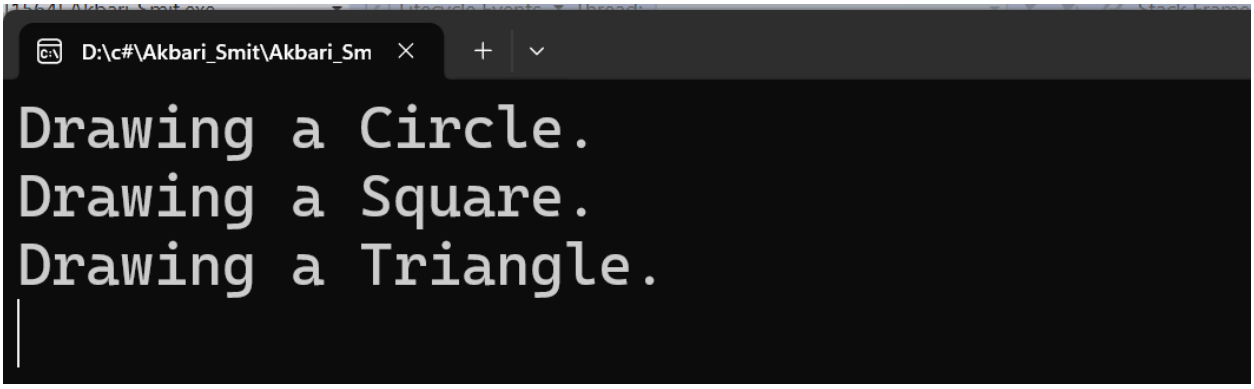


```
namespace Akbari_Smit
{
    /*Design a Shape class with a Draw() method and override it in Circle, Square, and
Triangle classes. Display overridden behavior using base class reference.*/
    6 references
    class Shape
    {
        6 references
        public virtual void Draw()
        {
            Console.WriteLine("Drawing a generic shape.");
        }
    }

    1 reference
    class Circle : Shape
    {
        4 references
        public override void Draw()
        {
            Console.WriteLine("Drawing a Circle.");
        }
    }

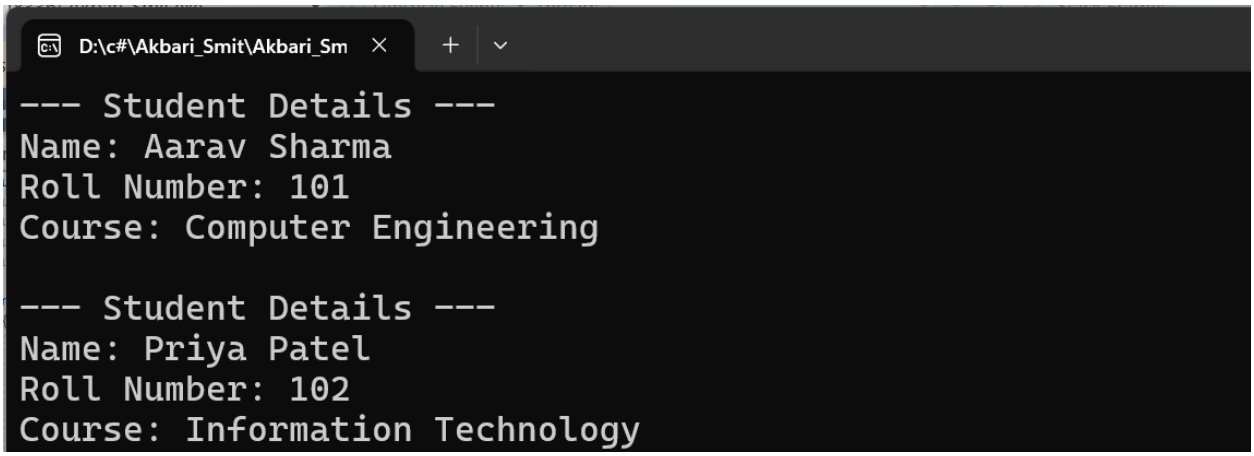
    1 reference
    class Square : Shape
    {
        4 references
        public override void Draw()
        {
            Console.WriteLine("Drawing a Square.");
        }
    }

    1 reference
    class Triangle : Shape
    {
        4 references
        public override void Draw()
        {
            Console.WriteLine("Drawing a Triangle.");
        }
    }
}
```



```
namespace Akbari_Smit
{
    //CollegeStudent. Display details using constructors.
    5 references
    class CollegeStudent
    {
        private string name;
        private int rollNumber;
        private string course;

        2 references
        public CollegeStudent(string snm, int sRoll, string sCourse)
        {
            name = snm;
            rollNumber = sRoll;
            course = sCourse;
        }
        2 references
        public void DisplayDetails()
        {
            Console.WriteLine("--- Student Details ---");
            Console.WriteLine("Name: " + name);
            Console.WriteLine("Roll Number: " + rollNumber);
            Console.WriteLine("Course: " + course);
            Console.WriteLine();
        }
    }
}
```

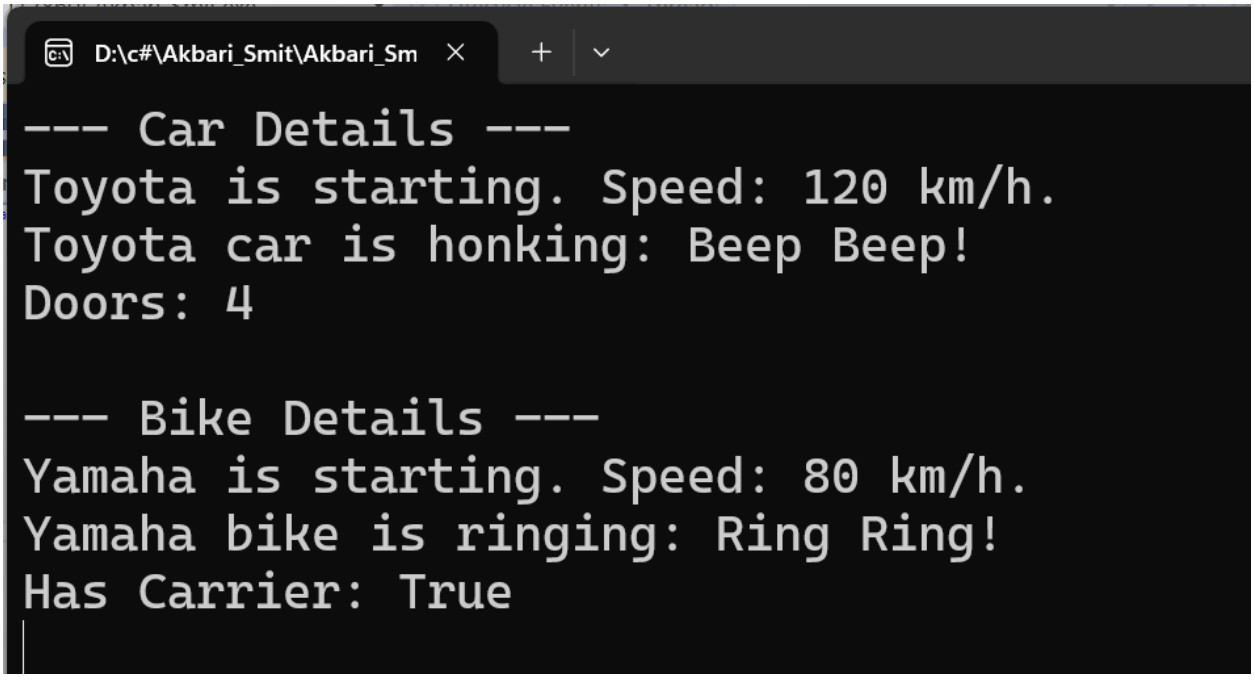


```
namespace Akbari_Smit
{
    /*Create a base class Vehicle and derive classes Car and Bike with addiϴonal members.
    Demonstrate access to base and derived members.*/
    2 references
    class Vehicle
    {
        public string brand;
        public int speed;

        2 references
        public void Start()
        {
            Console.WriteLine($"{brand} is starting. Speed: {speed} km/h.");
        }
    }
    2 references
    class Car : Vehicle
    {
        public int numberOfDoors;

        1 reference
        public void Honk()
        {
            Console.WriteLine($"{brand} car is honking: Beep Beep!");
        }
    }
    2 references
    class Bike : Vehicle
    {
        public bool hasCarrier;

        1 reference
        public void RingBell()
        {
            Console.WriteLine($"{brand} bike is ringing: Ring Ring!");
        }
    }
}
```



```
namespace Akbari_Smit
{
    /*Implement hierarchical inheritance with a base class Animal and derived classes Dog,
Cat, Cow. Include a method Sound() in each.*/
    6 references
    class Animal
    {
        6 references
        public virtual void Sound()
        {
            Console.WriteLine("Animal makes a sound.");
        }
    }
    1 reference
    class Dog : Animal
    {
        4 references
        public override void Sound()
        {
            Console.WriteLine("Dog barks: Woof Woof!");
        }
    }
    1 reference
    class Cat : Animal
    {
        4 references
        public override void Sound()
        {
            Console.WriteLine("Cat meows: Meow Meow!");
        }
    }
    1 reference
    class Cow : Animal
    {
        4 references
        public override void Sound()
        {
            Console.WriteLine("Cow moos: Moo Moo!");
        }
    }
}
```

```
D:\c#\Akbari_Smit\Akbari_Sm
--- Hierarchical Inheritance ---
Dog barks: Woof Woof!
Cat meows: Meow Meow!
Cow moos: Moo Moo!
```

```csharp
namespace Akbari_Smit
{
    /*Create a class BankAccount and a subclass SavingsAccount that adds interest. Show
method reuse and extension through inheritance.*/
    3 references
    class BankAccount
    {
        public string accnum;
        public double balance;

        1 reference
        public BankAccount(string accNum, double initialBalance)
        {
            accnum = accNum;
            balance = initialBalance;
        }
        1 reference
        public void Deposit(double amount)
        {
            balance += amount;
            Console.WriteLine($"Deposited: {amount:C}. New Balance: {balance:C}");
        }

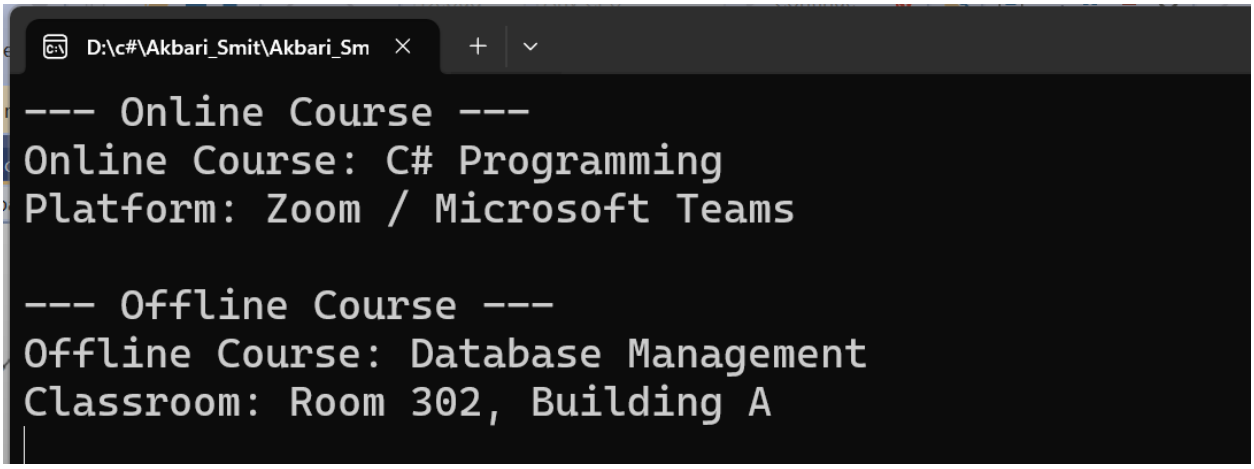
        1 reference
        public void DisplayInfo()
        {
            Console.WriteLine($"Account Number: {accnum}, Balance: {balance:C}");
        }
    }

    3 references
    class SavingsAccount : BankAccount
    {
        public double interestRate;
        1 reference
        public SavingsAccount(string accNum, double initialBalance, double rate)
            : base(accNum, initialBalance)
        {
            interestRate = rate;
        }
        1 reference
        public void AddInterest()
        {
            double interest = balance * (interestRate / 100);
            balance += interest;
            Console.WriteLine($"Interest Added ({interestRate}%): {interest:C}. Total Balance: {balance:C}");
        }
    }
```

```
D:\c#\Akbari_Smit\Akbari_Sm
--- Account Operations ---
Account Number: SA-501, Balance: $10,000.00
Deposited: $2,500.00. New Balance: $12,500.00
Interest Added (4.5%): $562.50. Total Balance: $13,062.50
```

```
namespace Akbari_Smit
{
    /*Use inheritance to model an educaϴonal system: Course as a base class
and OnlineCourse and OfflineCourse as derived classes. Include a method
CourseDetails().*/
    2 references
    class Course
    {
        public string courseName;

        4 references
        public virtual void CourseDetails()
        {
            Console.WriteLine($"Course Name: {courseName}");
        }
    }
    2 references
    class OnlineCourse : Course
    {
        public string platform;
        2 references
        public override void CourseDetails()
        {
            Console.WriteLine($"Online Course: {courseName}");
            Console.WriteLine($"Platform: {platform}");
        }
    }
    2 references
    class OfflineCourse : Course
    {
        public string classroom;
        2 references
        public override void CourseDetails()
        {
            Console.WriteLine($"Offline Course: {courseName}");
            Console.WriteLine($"Classroom: {classroom}");
        }
```



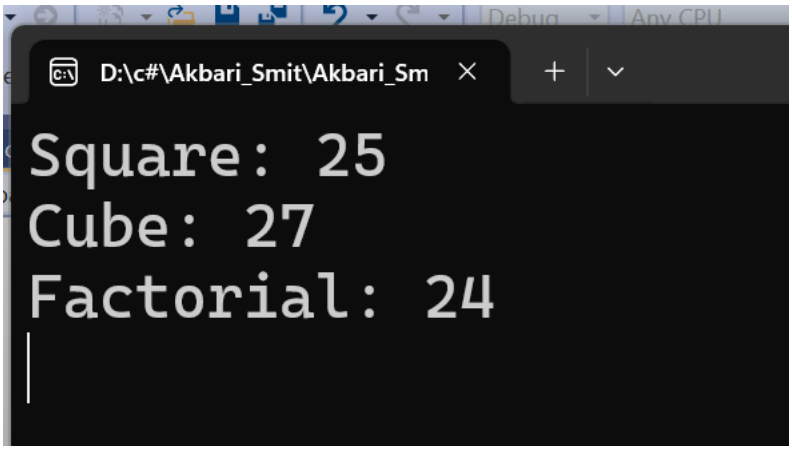
```
namespace Akbari_Smit
{

    2 references
    public sealed class MathOperations
    {
        //Create a sealed class Math Operations that contains uθlity methods like Square(), Cube(), and Factorial().
        1 reference
        public double Square(double number)
        {
            return number * number;
        }

        1 reference
        public double Cube(double number)
        {
            return number * number * number;
        }

        1 reference
        public long Factorial(int n)
        {
            if (n < 0)
            {
                throw new ArgumentOutOfRangeException(nameof(n), "Factorial is not defined for negative numbers.");
            }

            long result = 1;
            for (int i = 2; i <= n; i++)
            {
                result *= i;
            }
            return result;
        }
    }
}
```

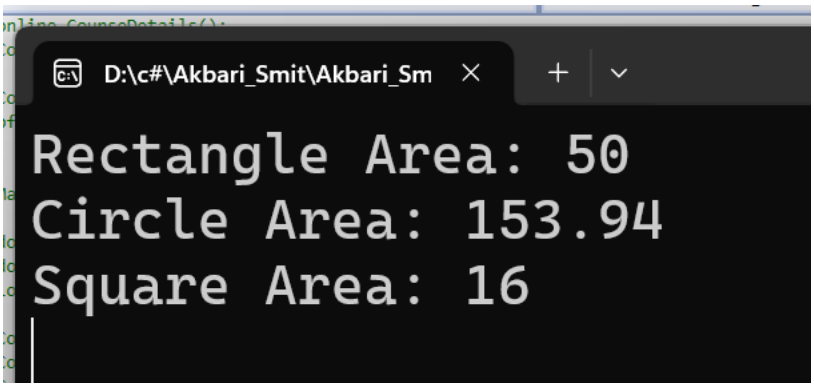


```
namespace Akbari_Smit
{
    /*Create an abstract class Shape with an abstract method
      CalculateArea() and implement it in Rectangle, Circle,
      and Square classes.*/
    6 references
    public abstract class Shapes
    {
        6 references
        public abstract double CalculateArea();
    }
    2 references
    public class Rectangle : Shapes
    {
        2 references
        public double Length { get; set; }
        2 references
        public double Width { get; set; }

        1 reference
        public Rectangle(double length, double width)
        {
            Length = length;
            Width = width;
        }
        4 references
        public override double CalculateArea()
        {
            return Length * Width;
        }
    }
    2 references
    public class Circles : Shapes
    {
        3 references
        public double Radius { get; set; }

        1 reference
        public Circles(double radius)
        {
            Radius = radius;
        }
        4 references
        public override double CalculateArea()
        {
            return Math.PI * Radius * Radius;
        }
    }
    2 references
    public class Squares : Shapes
    {
        3 references
        public double Side { get; set; }

        1 reference
        public Squares(double side)
        {
            Side = side;
        }
        4 references
        public override double CalculateArea()
        {
            return Side * Side;
        }
    }
```



```
using System.Threading.Tasks;

namespace Akbari_Smit
{
    /* Develop an abstract class Animal with properties
        and methods; override behaviors in Bird and Fish.*/

    7 references
    public abstract class Animals
    {
        7 references
        public string Name { get; set; }
        3 references
        public int Age { get; set; }

        2 references
        protected Animals(string name, int age)
        {
            Name = name;
            Age = age;
        }
        4 references
        public abstract void MakeSound();
        4 references
        public abstract void Move();
    }
    2 references
    public class Bird : Animals
    {
        1 reference
        public Bird(string name, int age) : base(name, age) { }

        3 references
        public override void MakeSound()
        {
            Console.WriteLine($"{Name} the bird chirps: Churp! Churp!");
        }

        3 references
```

```
2 references
public class Bird : Animals
{
    1 reference
    public Bird(string name, int age) : base(name, age) { }

    3 references
    public override void MakeSound()
    {
        Console.WriteLine($"{Name} the bird chirps: Churp! Churp!");
    }

    3 references
    public override void Move()
    {
        Console.WriteLine($"{Name}is flying in the sky high.");
    }
}
2 references
public class Fish : Animals
{
    1 reference
    public Fish(string name, int age) : base(name, age) { }

    3 references
    public override void MakeSound()
    {
        Console.WriteLine($"{Name} the fish makes no sound, just bubbles.");
    }

    3 references
    public override void Move()
    {
        Console.WriteLine($"{Name} is swimming gracefully in the water.");
    }
}
}
```

```
D:\c#\Akbari_Smit\Akbari_Sm
--- Parrot (2 years old) ---
Parrot the bird chirps: Churp! Churp!
Parrotis flying in the sky high.
--- Goldfish (1 year old) ---
Goldfish the fish makes no sound, just bubbles.
Goldfish is swimming gracefully in the water.
```

```
namespace Akbari_Smit
{
    /*Create an interface IPayable with method CalculatePayment().
     Implement it in Invoice and Employee classes.*/

    4 references
    pu namespace Akbari_Smit
           public interface IPayable
    {
        decimal CalculatePayment();
    }
    2 references
    public class Invoice : IPayable
    {
        1 reference
        public string ItemName { get; set; }
        2 references
        public decimal Price { get; set; }
        2 references
        public int Quantity { get; set; }

        1 reference
        public Invoice(string itemName, decimal price, int quantity)
        {
            ItemName = itemName;
            Price = price;
            Quantity = quantity;
        }

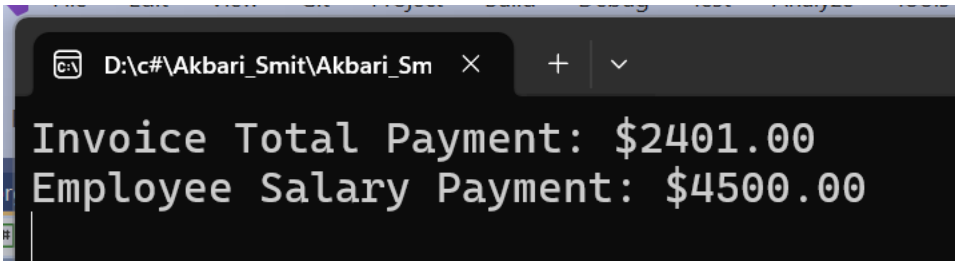
        3 references
        public decimal CalculatePayment()
        {
            return Price * Quantity;
        }
    }
    2 references
    public class Employees : IPayable
```

```
        public Invoice(string itemName, decimal price, int quantity)
        {
            ItemName = itemName;
            Price = price;
            Quantity = quantity;
        }

        3 references
        public decimal CalculatePayment()
        {
            return Price * Quantity;
        }
    }
    2 references
    public class Employees : IPayable
    {
        1 reference
        public string Name { get; set; }
        2 references
        public decimal MonthlySalary { get; set; }

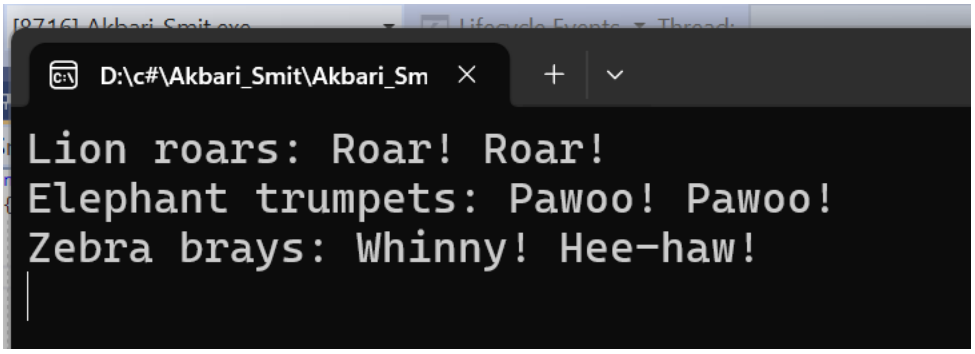
        1 reference
        public Employees(string name, decimal monthlySalary)
        {
            Name = name;
            MonthlySalary = monthlySalary;
        }

        3 references
        public decimal CalculatePayment()
        {
            return MonthlySalary;
        }
```



```
namespace Akbari_Smit
{
    /*Create a real-world example using interface IAnimal and
      implement Sound() method in classes Lion, Elephant, and Zebra.*/

    6 references
    public interface IAnimal
    {
        6 references
        void Sound();
    }
    1 reference
    public class Lion : IAnimal
    {
        4 references
        public void Sound()
        {
            Console.WriteLine("Lion roars: Roar! Roar!");
        }
    }
    1 reference
    public class Elephant : IAnimal
    {
        4 references
        public void Sound()
        {
            Console.WriteLine("Elephant trumpets: Pawoo! Pawoo!");
        }
    }
    1 reference
    public class Zebra : IAnimal
    {
        4 references
        public void Sound()
        {
            Console.WriteLine("Zebra brays: Whinny! Hee-haw!");
        }
    }
}
```



```
namespace Akbari_Smit
{
    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            prg1 p1 = new prg1();
            p1.prg();
            prg2 p2 = new prg2();
            p2.prg();
            prg3 p3 = new prg3();
            p3.prg();
            prg4 p4 = new prg4();
            p4.prg();
            prg5 p5 = new prg5();
            p5.prg();
            prg6 p6 = new prg6();
            p6.prg();
            prg7 p7 = new prg7();
            p7.prg();
            prg8 p8 = new prg8();
            p8.prg();
            prg9 p9 = new prg9();
            p9.prg();
            prg10 p = new prg10();
            int a = 20;
            int b = 30;
            p.prg(ref a, ref b);

            item i = new item();
            i.getitem();
            i.putitem();

            distances d1 = new distances();
            d1.display();

            distances d2 = new distances(888);
            d2.display();


            Console.Write("Enter name: ");
            string name = Console.ReadLine();

            Console.Write("Enter address: ");
            string address = Console.ReadLine();

            Console.Write("Enter phone number: ");
            string phone = Console.ReadLine();

            person person = new person(name, address, phone);

            person.Display();


            Console.Write("Enter name: ");
            string names = Console.ReadLine();

            Console.Write("Enter address: ");
            string addresss = Console.ReadLine();

            Console.Write("Enter phone number: ");
            string phones = Console.ReadLine();

            persons person1 = new persons(names, addresss, phones);

            person1.Display();
```

```
Student s1 = new Student();
s1.Display();

Student s2 = new Student("Alice", 20);
s2.Display();

Student s3 = new Student("Bob", 22, 'M');
s3.Display();


calculator c = new calculator();

int ans1 = c.sum(8, 8);
Console.WriteLine(ans1);

float ans2 = c.sum(8.8f, 8.8f);
Console.WriteLine(ans1);

float ans3 = c.sum(8, 5.5f);
Console.WriteLine(ans3);

float ans4 = c.sum(5.5f, 8);
Console.WriteLine(ans4);

printer pr = new printer();
pr.display('s');
pr.display();
pr.display('s', 55);
Console.Read();


ShapeAreaCalculator cc = new ShapeAreaCalculator();
// Square Calculation
Console.Write("Enter side of the square: ");
double side = Convert.ToDouble(Console.ReadLine());
Console.WriteLine("Area of Square: " + cc.Area(side));

// Circle Calculation
```

```
// Circle Calculation
Console.Write("Enter radius of the circle: ");
float radius = Convert.ToSingle(Console.ReadLine());
Console.WriteLine("Area of Circle: " + cc.Area(radius));

// Triangle Calculation
Console.Write("Enter base of the triangle: ");
double ba = Convert.ToDouble(Console.ReadLine());
Console.Write("Enter height of the triangle: ");
double h = Convert.ToDouble(Console.ReadLine());
Console.WriteLine("Area of Triangle: " + cc.Area(ba, h));


Employee emp = new Employee();
emp.DisplayInfo();
Console.WriteLine();


Manager mgr = new Manager();
mgr.DisplayInfo();
Console.WriteLine();


Employee polyEmp = new Manager();
polyEmp.DisplayInfo();


BasePrinter printer1 = new LaserPrinter();
BasePrinter printer2 = new InkjetPrinter();
BasePrinter printer3 = new BasePrinter();

printer1.Print();
printer2.Print();
printer3.Print();


Shape S1 = new Circle();
```

```
Shape S3 = new Triangle();

S1.Draw();
S2.Draw();
S3.Draw();



CollegeStudent student1 = new CollegeStudent("Aarav Sharma", 101, "Computer Engineering");
CollegeStudent student2 = new CollegeStudent("Priya Patel", 102, "Information Technology");

student1.DisplayDetails();
student2.DisplayDetails();

Car myCar = new Car();
myCar.brand = "Toyota";
myCar.speed = 120;
myCar.numberOfDoors = 4;

Console.WriteLine("--- Car Details ---");
myCar.Start();
myCar.Honk();
Console.WriteLine($"Doors: {myCar.numberOfDoors}\n");

Bike myBike = new Bike();
myBike.brand = "Yamaha";
myBike.speed = 80;
myBike.hasCarrier = true;
Console.WriteLine("--- Bike Details ---");
myBike.Start();
myBike.RingBell();
Console.WriteLine($"Has Carrier: {myBike.hasCarrier}");
```

```
Animal dog = new Dog();
Animal cat = new Cat();
Animal cow = new Cow();

Console.WriteLine("--- Hierarchical Inheritance ---");
dog.Sound();
cat.Sound();
cow.Sound();


SavingsAccount mySavings = new SavingsAccount("SA-501", 10000, 4.5);

Console.WriteLine("--- Account Operations ---");
mySavings.DisplayInfo();
mySavings.Deposit(2500);
mySavings.AddInterest();




OnlineCourse online = new OnlineCourse();
online.courseName = "C# Programming";
online.platform = "Zoom / Microsoft Teams";

OfflineCourse offline = new OfflineCourse();
offline.courseName = "Database Management";
offline.classroom = "Room 302, Building A";

Console.WriteLine("--- Online Course ---");
online.CourseDetails();
Console.WriteLine();

Console.WriteLine("--- Offline Course ---");
offline.CourseDetails();


MathOperations math = new MathOperations();
```

```
        MathOperations math = new MathOperations();

        double squareResult = math.Square(5);
        double cubeResult = math.Cube(3);
        long factorialResult = math.Factorial(4);

        Console.WriteLine($"Square: {squareResult}");
        Console.WriteLine($"Cube: {cubeResult}");
        Console.WriteLine($"Factorial: {factorialResult}");


        Shapes rect = new Rectangle(10, 5);
        Shapes circle = new Circles(7);
        Shapes square = new Squares(4);

        Console.WriteLine($"Rectangle Area: {rect.CalculateArea()}");
        Console.WriteLine($"Circle Area: {circle.CalculateArea():F2}");
        Console.WriteLine($"Square Area: {square.CalculateArea()}");



        Animals myBird = new Bird("Parrot", 2);
        Animals myFish = new Fish("Goldfish", 1);

        Console.WriteLine($"--- {myBird.Name} ({myBird.Age} years old) ---");
        myBird.MakeSound();
        myBird.Move();

        Console.WriteLine($"--- {myFish.Name} ({myFish.Age} year old) ---");
        myFish.MakeSound();
        myFish.Move();



        IPayable invoice = new Invoice("Laptop", 1200.50m, 2);
        IPayable employee = new Employees("John Doe", 4500.00m);


        Animals myBird = new Bird("Parrot", 2);
        Animals myFish = new Fish("Goldfish", 1);

        Console.WriteLine($"--- {myBird.Name} ({myBird.Age} years old) ---");
        myBird.MakeSound();
        myBird.Move();

        Console.WriteLine($"--- {myFish.Name} ({myFish.Age} year old) ---");
        myFish.MakeSound();
        myFish.Move();



        IPayable invoice = new Invoice("Laptop", 1200.50m, 2);
        IPayable employee = new Employees("John Doe", 4500.00m);

        Console.WriteLine($"Invoice Total Payment: ${invoice.CalculatePayment()}");
        Console.WriteLine($"Employee Salary Payment: ${employee.CalculatePayment()}");



        IAnimal lion = new Lion();
        IAnimal elephant = new Elephant();
        IAnimal zebra = new Zebra();

        lion.Sound();
        elephant.Sound();
        zebra.Sound();

        Console.ReadLine();
    }
}
```